

Universidade Federal de Santa Catarina · Centro Tecnológico
Departamento de Informática e Estatística · Ciência da Computação

Polyglot Import CSV

Uma solução para importação de documentos CSV para múltiplos bancos de dados

Lucas Bueno Cesario
Orientador: Prof. Dr. Ronaldo dos Santos Mello

Roteiro

1 Introdução

2 Fundamentação e trabalhos relacionados

3 Proposta

4 Avaliação

5 Conclusão

Roteiro

1 **Introdução**

2 Fundamentação e trabalhos relacionados

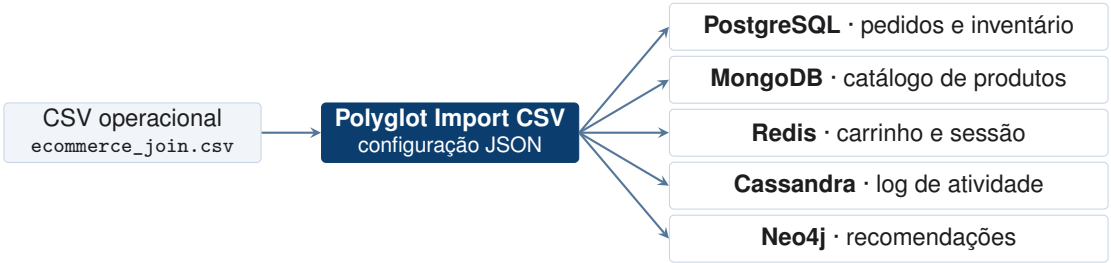
3 Proposta

4 Avaliação

5 Conclusão

Persistência poliglota

- Um só modelo de dados não atende todas as necessidades.
- **Persistência poliglota**: cada tipo de dado fica no SGBD (sistema gerenciador de banco de dados) mais adequado.



Cada dado do e-commerce no SGBD mais adequado. CSV: texto com valores separados por vírgula. JSON: texto estruturado.

O problema

- Cada SGBD tem o seu **importador nativo**, com sintaxe própria.
- Cada importador atende um único SGBD por vez.
- Nenhuma ferramenta encontrada importa o mesmo CSV nos cinco.

Importadores nativos de CSV

	PostgreSQL	Redis	MongoDB	Cassandra	Neo4j
Comando	`COPY`	`redis-cli --pipe`	`mongoimport`	`cqlsh` e `sstableloader`	`neo4j-admin import`
Detalhe	colunas na ordem do CSV	arquivo em protocolo bruto	tipo e cabeçalho por parâmetros	tipos e colunas iguais aos da tabela	CSV separado para nós e relacionamentos

Objetivos

Objetivo geral

Desenvolver a ferramenta *Polyglot Import CSV*, que importa CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

Objetivos específicos

1. **Configuração** em JSON (texto estruturado): filtros, chave primária e nome da coluna.
2. **Validação prévia**: um erro na configuração impede o início da importação.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).

Roteiro

1 Introdução

2 Fundamentação e trabalhos relacionados

3 Proposta

4 Avaliação

5 Conclusão

Cinco paradigmas, cinco SGBDs

Relacional
PostgreSQL

Tabelas ligadas por chaves, com transações consistentes.

Exemplo: pedidos e inventário.

Chave-valor
Redis

Cada chave aponta para um valor.

Exemplo: carrinho e sessão.

Documento
MongoDB

Documentos JSON com campos variáveis.

Exemplo: catálogo de produtos.

Colunar
Cassandra

Linhas com famílias de colunas.

Exemplo: log de atividade.

Grafo
Neo4j

Nós e arestas com propriedades.

Exemplo: recomendação de produtos.

Os quatro últimos são os bancos **NoSQL** (não relacionais).
Um SGBD **multimodelo** reúne vários desses modelos no mesmo sistema.

Trabalhos relacionados

Comparação com os trabalhos relacionados ✓ o Cap. 3 afirma — o Cap. 3 não afirma

Trabalho	Foco	Destinos	Entrada CSV	Mapeamento declarativo validado	Revisão sem gravar
Importadores nativos	importação	um SGBD por vez	✓[1]	—	—
ArangoDB e `arangoimport`	importação	um SGBD multimodelo	✓	—	—
dbcrossbar	migração	PostgreSQL, BigQuery, nuvem	✓	—	—
Hackolade	modelagem visual	dezenas de alvos	—	—	—
Frameworks de acesso[2]	acesso em tempo de execução	vários SGBDs	—	—	—
Polystores[3] (BigDAWG)	consulta federada	fontes heterogêneas	—	—	—

[1] Redis: arquivo em protocolo bruto.
[2] Interface para ler e gravar em vários SGBDs (PEREIRA; GUERRA; ROSA, 2025; SACHDEVA; VASAVA, 2024).
[3] Uma só consulta sobre vários SGBDs (consulta federada).

Trabalhos relacionados

Comparação com os trabalhos relacionados ✓ o Cap. 3 afirma — o Cap. 3 não afirma

Trabalho	Foco	Destinos	Entrada CSV	Mapeamento declarativo validado	Revisão sem gravar
Importadores nativos	importação	um SGBD por vez	✓[1]	—	—
ArangoDB e `arangoimport`	importação	um SGBD multimodelo	✓	—	—
dbcrossbar	migração	PostgreSQL, BigQuery, nuvem	✓	—	—
Hackolade	modelagem visual	dezenas de alvos	—	—	—
Frameworks de acesso[2]	acesso em tempo de execução	vários SGBDs	—	—	—
Polystores[3] (BigDAWG)	consulta federada	fontes heterogêneas	—	—	—
Polyglot Import CSV	**importação**	**cinco SGBDs, cinco paradigmas**	✓	✓ **(JSON Schema)**	✓ **(`--dry-run`)**

[1] Redis: arquivo em protocolo bruto.
[2] Interface para ler e gravar em vários SGBDs (PEREIRA; GUERRA; ROSA, 2025; SACHDEVA; VASAVA, 2024).
[3] Uma só consulta sobre vários SGBDs (consulta federada).

Roteiro

1 Introdução
2 Fundamentação e trabalhos relacionados
3 Proposta
4 Avaliação
5 Conclusão

Visão geral

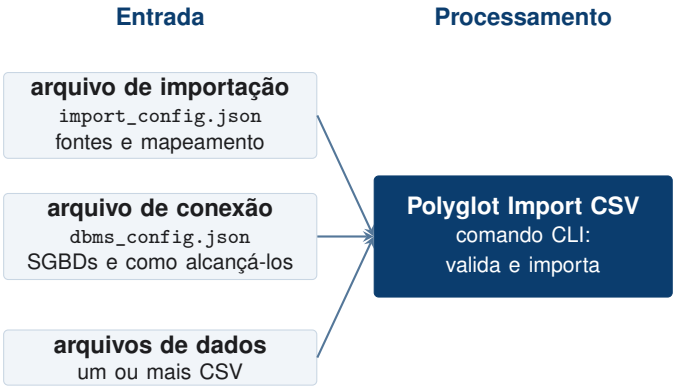
Entrada

arquivo de importação
`import_config.json`
fontes e mapeamento

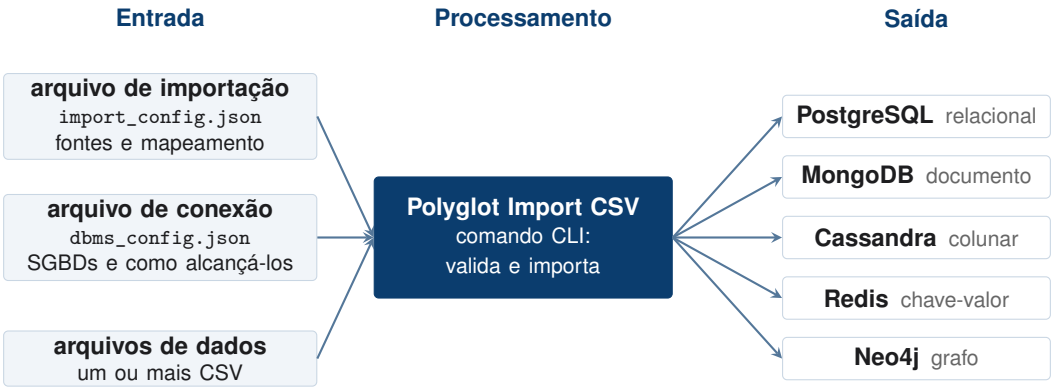
arquivo de conexão
`dbms_config.json`
SGBDs e como alcançá-los

arquivos de dados
um ou mais CSV

Visão geral


Entrada
Processamento
arquivo de importação
import_config.json
fontes e mapeamento
arquivo de conexão
dbms_config.json
SGBDs e como alcançá-los
arquivos de dados
um ou mais CSV
Polyglot Import CSV
comando CLI:
valida e importa

Visão geral



O comando valida os arquivos de configuração e só então importa para os SGBDs listados neles.

Arquivo de importação

Arquivo de importação: JSON com as fontes (CSVs) e o que gravar em cada SGBD.

Trecho de `import_config.json`: a fonte `stock` alimenta a tabela `inventory`.

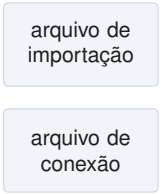
```
"sources": {
  "stock": "ecommerce_stock.csv",
  "purchase": "ecommerce_purchase.csv", ...
},
"postgres": {
  "entities": { ...
    "inventory": {
      "source": "stock",
      "columns": {
        "product_id": { "is_key": true, "db_type": "BIGINT" },
        "quantity_available": { "db_type": "BIGINT" }, ...
      } }, ...
  }, ...
}, ...
```

- `sources`: dá um nome a cada CSV.
- `postgres`: um bloco por SGBD de destino.
- `entities`: tabelas, coleções, nós...
- `columns`: colunas, com chave (`is_key`) e tipo (`db_type`).

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

Antes de gravar, a validação passa por **três camadas**:

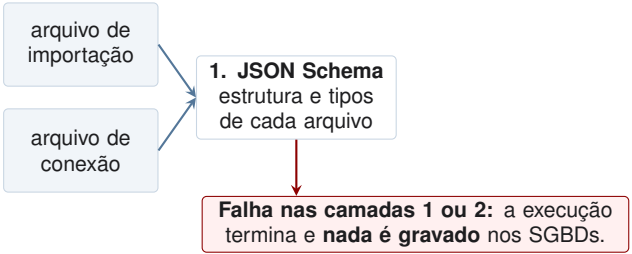


Fluxo da validação prévia, dos arquivos de configuração à importação.

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

Antes de gravar, a validação passa por **três camadas**:

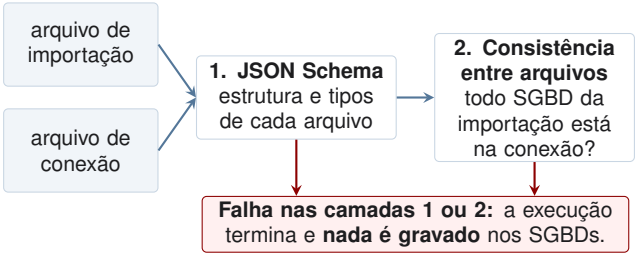


Fluxo da validação prévia, dos arquivos de configuração à importação.

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

Antes de gravar, a validação passa por **três camadas**:

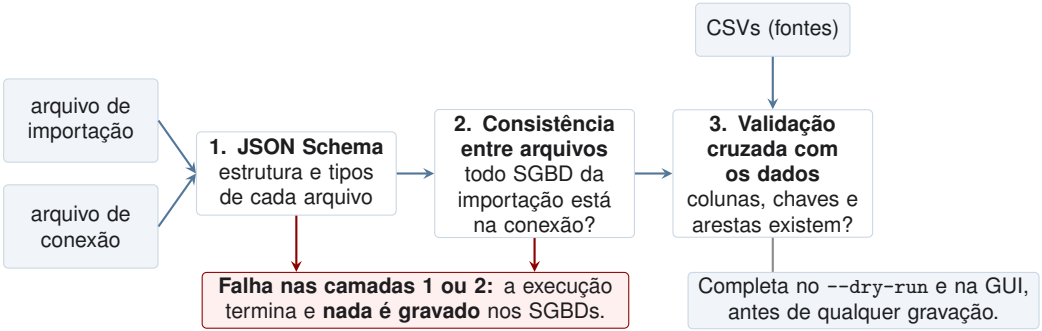


Fluxo da validação prévia, dos arquivos de configuração à importação.

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

Antes de gravar, a validação passa por **três camadas**:

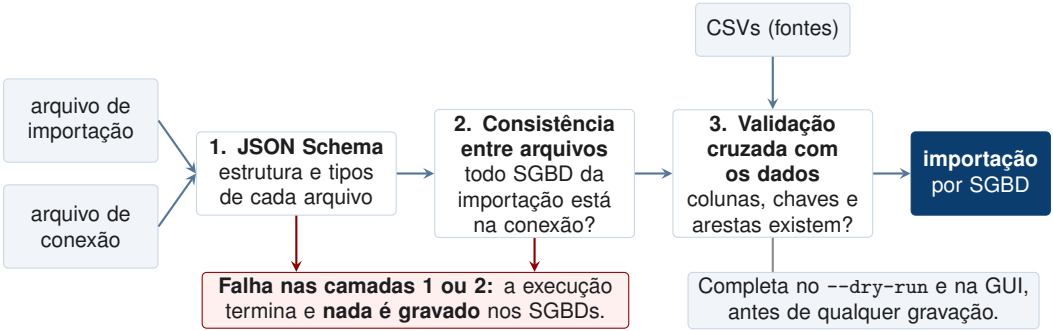


Fluxo da validação prévia, dos arquivos de configuração à importação.

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

Antes de gravar, a validação passa por **três camadas**:



Fluxo da validação prévia, dos arquivos de configuração à importação.

Cenário e-commerce

Tipos de evento do cenário de e-commerce todo evento registra o instante e o usuário envolvido

Tipo	Evento	Demais dados registrados
`stock`	reposição de estoque	produto, categoria, quantidade disponível e preço
`purchase`	compra	pedido, produto, contraparte da venda, pagamento e avaliação
`select_product`	seleção de produto	produto selecionado
`add_to_cart`	adição ao carrinho	carrinho, produto e quantidade

Cenário e-commerce

Tipos de evento do cenário de e-commerce todo evento registra o instante e o usuário envolvido

Tipo	Evento	Demais dados registrados
`stock`	reposição de estoque	produto, categoria, quantidade disponível e preço
`purchase`	compra	pedido, produto, contraparte da venda, pagamento e avaliação
`select_product`	seleção de produto	produto selecionado
`add_to_cart`	adição ao carrinho	carrinho, produto e quantidade

Multifonte um CSV por tipo de evento

```
"sources": {
  "stock": "ecommerce_stock.csv",
  "purchase": "ecommerce_purchase.csv", ...
}
```

Combinado um CSV com coluna de origem

```
"sources": {
  "ecommerce": {
    "file": "ecommerce_join.csv",
    "origin_column": true } }
```

Nos dois modos, o resto do arquivo de importação é igual. O exemplo usa o multifonte.

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory

product_id	2
quantity_available	2
last_restock_date	02/11/2023 03:30
price	50.5

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory

product_id	2
quantity_available	2
last_restock_date	02/11/2023 03:30
price	50.5

MongoDB
documento

← stock
product_catalog

```
{"product_id": "2",
 ...,
 "category": {...},
 "stock": {
  "quantity_available":
   "2", ...}}
```

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id 7fd7...`) e `Product2` (`product_id 2`) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.5

MongoDB
documento

← stock
product_catalog
{"product_id": "2",
...,
"category": {...},
"stock": {
"quantity_available":
"2", ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time
event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id 7fd7...`) e `Product2` (`product_id 2`) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.5

MongoDB
documento

← stock
product_catalog
{"product_id": "2",
...,
"category": {...},
"stock": {
"quantity_available":
"2", ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

Redis
chave-valor

← add_to_cart
shopping_cart
chave user:7fd7...:cart
valor {...,
"cart_product_id": "3",
"cart_quantity": "1"}

← select_product
user_session
chave 7fd7...
valor {"user_name":
"user1", ...}

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.5

MongoDB
documento

← stock
product_catalog
{"product_id": "2",
...,
"category": {...},
"stock": {
"quantity_available":
"2", ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

Redis
chave-valor

← add_to_cart
shopping_cart
chave user:7fd7...:cart
valor {...,
"cart_product_id": "3",
"cart_quantity": "1"}

← select_product
user_session
chave 7fd7...
valor {"user_name":
"user1", ...}

Neo4j
grafo

← stock e purchase
(:User {...,
user_name: "user1",
...})
-[:PURCHASED {
order_number:
"ORD1", ...}]->
(:Product {
product_id: "2", ...})

Algoritmo de execução

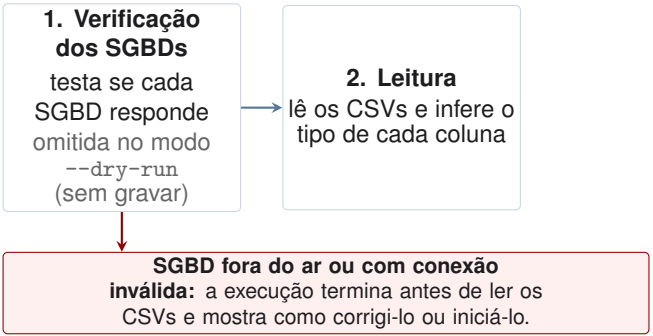
Com as duas configurações validadas, a importação segue **quatro etapas**:

1. Verificação dos SGBDs
testa se cada SGBD responde
omitida no modo `--dry-run` (sem gravar)

SGBD fora do ar ou com conexão inválida: a execução termina antes de ler os CSVs e mostra como corrigi-lo ou iniciá-lo.

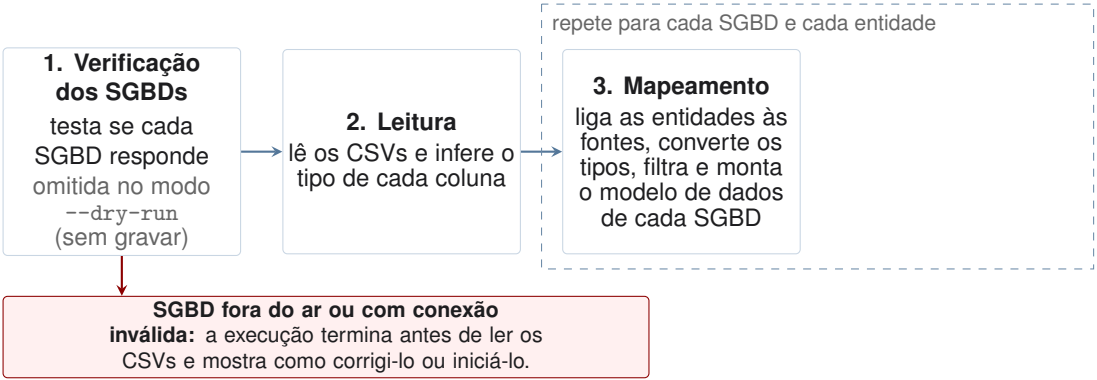
Algoritmo de execução

Com as duas configurações validadas, a importação segue **quatro etapas**:



Algoritmo de execução

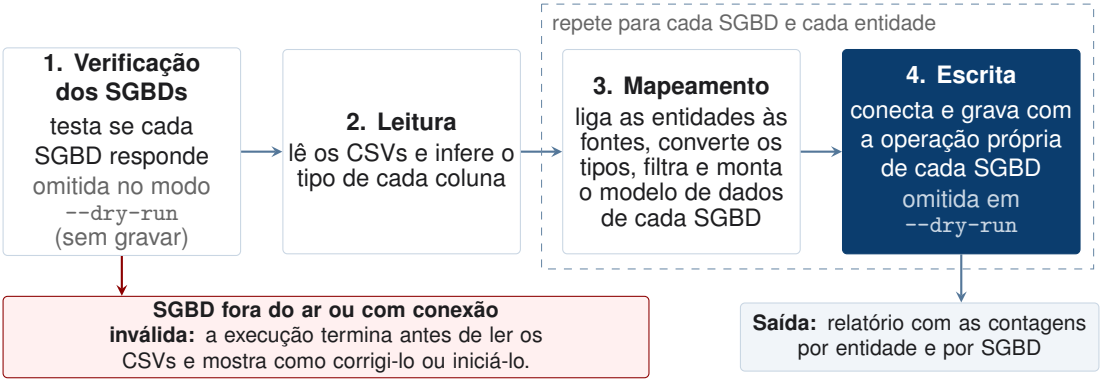
Com as duas configurações validadas, a importação segue **quatro etapas**:



Etapas da execução; o pontilhado marca o que se repete por SGBD e por entidade.

Algoritmo de execução

Com as duas configurações validadas, a importação segue **quatro etapas**:



Etapas da execução; o pontilhado marca o que se repete por SGBD e por entidade.

Exemplo de execução

```
python -m polyglotimportcsv \                          ← executa a ferramenta
  --config data/ecommerce/import_config.json \         ← arquivo de importação
  --dbms-config data/ecommerce/dbms_config.json \      ← arquivo de conexão
  --dry-run                                            ← valida e conta, sem gravar
```

Exemplo de execução

```
python -m polyglotimportcsv \                          ← executa a ferramenta
  --config data/ecommerce/import_config.json \         ← arquivo de importação
  --dbms-config data/ecommerce/dbms_config.json \      ← arquivo de conexão
  --dry-run                                            ← valida e conta, sem gravar
```

Contagens reportadas pelo modo `--dry-run` no cenário e-commerce — após projeção/deduplicação

SGBD	Destino	Registros
PostgreSQL	`categories`	8
PostgreSQL	`products`	8
PostgreSQL	`inventory`	8
PostgreSQL	`orders`	8
MongoDB	`product_catalog`	8 documentos
Cassandra	`user_activity_log`	32 linhas (união das 4 fontes)

SGBD	Destino	Registros
Redis	`shopping_cart`	8
Redis	`user_session`	8
Neo4j	nó `User`	8
Neo4j	nó `Product`	8
Neo4j	aresta `PURCHASED`	8 (uma por linha da fonte `purchase`)

Escrita e execução

Estratégia de escrita como os valores viram tipos nativos e são gravados

ingênua (*naive*)
primeira versão, linha de base

Converte cada célula e grava cada linha com um comando próprio.

×

em lote (*batch*)
estratégia atual da ferramenta

Converte uma coluna inteira de uma vez e grava muitos registros por comando.

Escrita e execução

Estratégia de escrita como os valores viram tipos nativos e são gravados

ingênua (*naive*)
primeira versão, linha de base

Converte cada célula e grava cada linha com um comando próprio.

×

em lote (*batch*)
estratégia atual da ferramenta

Converte uma coluna inteira de uma vez e grava muitos registros por comando.

Modelo de execução (`--execution`) quando os dados ocupam a memória

materialização
`materialize`

Carrega a fonte inteira na memória e só então processa. O pico de memória cresce com o tamanho do arquivo.

×

fluxo (*streaming*)
`stream`, o padrão em importações reais

Lê o arquivo em blocos e grava cada bloco antes de ler o próximo. O pico de memória fica perto de um bloco.

Escrita e execução

Estratégia de escrita como os valores viram tipos nativos e são gravados

ingênua (*naive*)
primeira versão, linha de base

Converte cada célula e grava cada linha com um comando próprio.

×

em lote (*batch*)
estratégia atual da ferramenta

Converte uma coluna inteira de uma vez e grava muitos registros por comando.

Modelo de execução (`--execution`) quando os dados ocupam a memória

materialização
`materialize`

Carrega a fonte inteira na memória e só então processa. O pico de memória cresce com o tamanho do arquivo.

×

fluxo (*streaming*)
`stream`, o padrão em importações reais

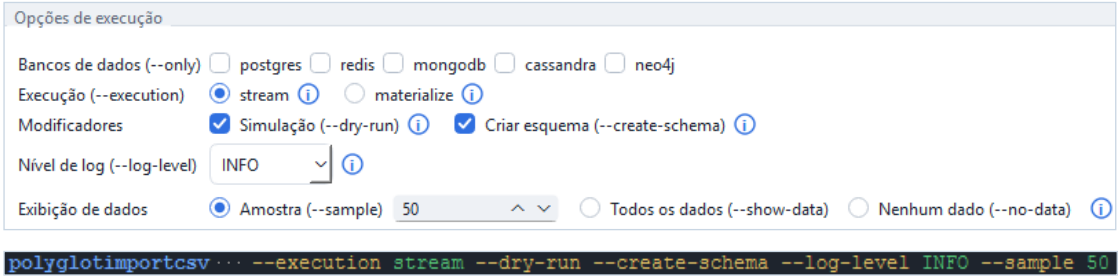
Lê o arquivo em blocos e grava cada bloco antes de ler o próximo. O pico de memória fica perto de um bloco.

Os dados gravados são os mesmos nos dois casos. Muda o custo, que a avaliação mede.

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela

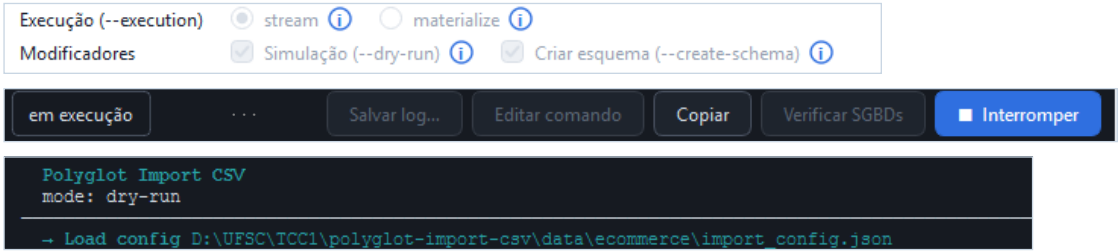


Ocioso: um controle para cada opção de uso cotidiano, com ícone de ajuda, e o comando montado, colorido e somente leitura.

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela

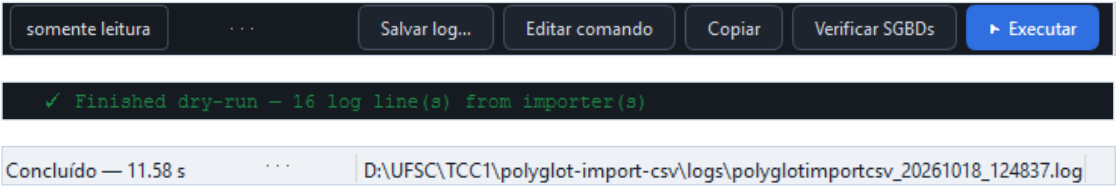


Executando: formulário bloqueado, botão de interrupção e saída colorida da CLI chegando ao console em tempo real.

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela

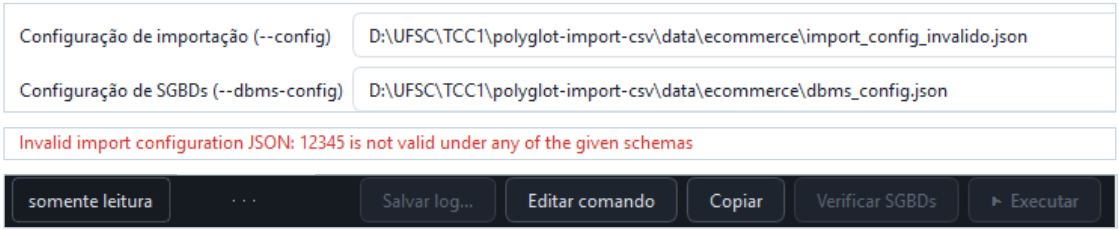


Concluído: desfecho e arquivo de log da sessão na barra de estado.

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela

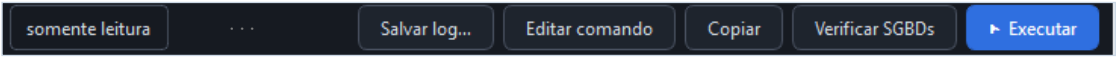


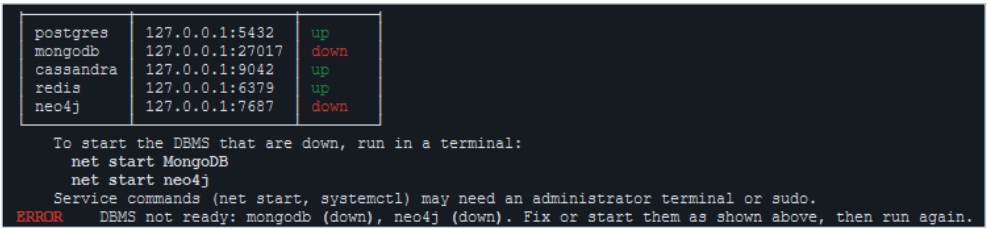
Erro antes de executar: o arquivo de importação viola o *JSON Schema*; a mensagem é a da CLI e o botão Executar fica desabilitado.

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela





Verificação dos SGBDs: MongoDB e Neo4j fora do ar e, abaixo da tabela de estados, os comandos que os iniciam.

Roteiro

1 Introdução
2 Fundamentação e trabalhos relacionados
3 Proposta
4 Avaliação
5 Conclusão

Metodologia da avaliação

Dados sintéticos. Gerador determinístico: a mesma escala N gera sempre os mesmos arquivos.

Volumes. N = 1.000, 10.000 e 100.000 linhas.

Fonte: §5.1, §5.2 e Tabela 13 do relatório.

Metodologia da avaliação

Dados sintéticos. Gerador determinístico: a mesma escala N gera sempre os mesmos arquivos.

Volumes. N = 1.000, 10.000 e 100.000 linhas.

Repetições. Três execuções por configuração, com os SGBDs limpos antes; vale a mediana (o valor do meio).

Duas campanhas. A instrumentação de memória rastreia cada alocação do Python e distorce os tempos.
Tempo: campanha sem instrumentação.
Pico de memória (maior memória alocada): campanha com instrumentação.

Precisão. Tempo: cerca de 20%; memória: $< 0{,}1\%$.

Fonte: §5.1, §5.2 e Tabela 13 do relatório.

Metodologia da avaliação

Dados sintéticos. Gerador determinístico: a mesma escala N gera sempre os mesmos arquivos.

Volumes. N = 1.000, 10.000 e 100.000 linhas.

Repetições. Três execuções por configuração, com os SGBDs limpos antes; vale a mediana (o valor do meio).

Duas campanhas. A instrumentação de memória rastreia cada alocação do Python e distorce os tempos.
Tempo: campanha sem instrumentação.
Pico de memória (maior memória alocada): campanha com instrumentação.

Precisão. Tempo: cerca de 20%; memória: $< 0{,}1\%$.

Fonte: §5.1, §5.2 e Tabela 13 do relatório.

Ambiente: uma máquina
Windows 11, 8 GB de memória

Em Docker: PostgreSQL 16, Redis 7, MongoDB 7, Neo4j 5 e Cassandra 4.1, este com 1 GB de memória Java (*heap*).

Metodologia da avaliação

Dados sintéticos. Gerador determinístico: a mesma escala *N* gera sempre os mesmos arquivos.

Volumes. *N* = 1.000, 10.000 e 100.000 linhas.

Repetições. Três execuções por configuração, com os SGBDs limpos antes; vale a mediana (o valor do meio).

Duas campanhas. A instrumentação de memória rastreia cada alocação do Python e distorce os tempos.
Tempo: campanha sem instrumentação.
Pico de memória (maior memória alocada): campanha com instrumentação.

Precisão. Tempo: cerca de 20%; memória: $< 0{,}1\%$.

Fonte: §5.1, §5.2 e Tabela 13 do relatório.

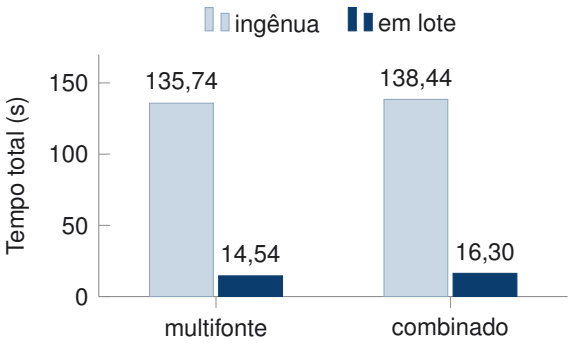
Ambiente: uma máquina
Windows 11, 8 GB de memória

Em Docker: PostgreSQL 16, Redis 7, MongoDB 7, Neo4j 5 e Cassandra 4.1, este com 1 GB de memória Java (*heap*).

Questões de pesquisa (QP)

QP1 Custo por SGBD
QP2 Modos de importação
QP3 Escalabilidade
QP4 Estratégia de escrita
QP5 Modelo de execução
QP6 Memória limitada

Em lote: cerca de 9× mais rápida (QP4)



~9×

mais rápida que a ingênua

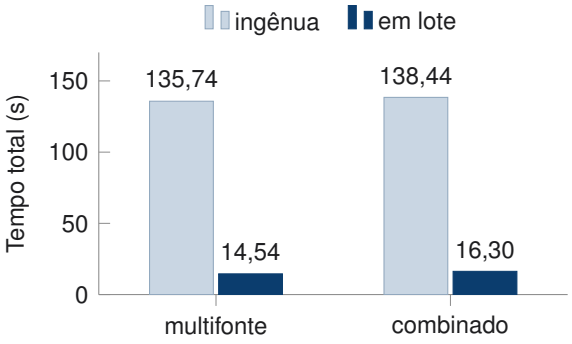
multifonte	9,3×
combinado	8,5×

as mesmas 27.494 linhas gravadas

Tempo total de importação por estratégia de escrita.

Fonte: Tabela 14 e Figura 17 do relatório (redesenhada em escala linear); N = 10.000, materialização; campanha sem instrumentação de memória.

Em lote: cerca de 9× mais rápida (QP4)



Tempo total de importação por estratégia de escrita.

~9×

mais rápida que a ingênua

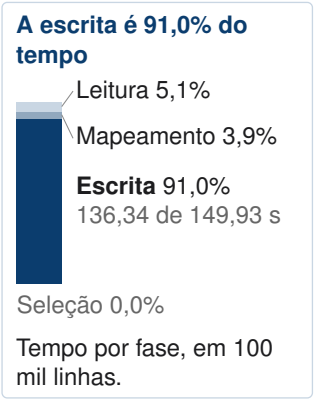
multifonte	9,3×
combinado	8,5×

as mesmas 27.494 linhas gravadas

Onde está o ganho (multifonte): mapeamento, cerca de 42× (conversão vetorizada: a coluna inteira de uma vez); escrita, 7,7× (gravação em lote).

Fonte: Tabela 14 e Figura 17 do relatório (redesenhada em escala linear); $N = 10.000$, materialização; campanha sem instrumentação de memória.

A escrita domina o tempo (QP1, QP3)



Fonte: Tabela 15, Figura 18 e Figura 20 do relatório; multifonte, em lote; fases e vazão com $N = 100.000$ e materialização; campanha sem instrumentação de memória.

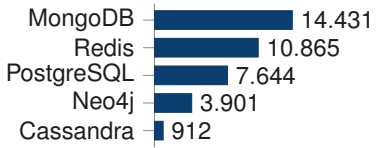
A escrita domina o tempo (QP1, QP3)

A escrita é 91,0% do tempo



Tempo por fase, em 100 mil linhas.

A vazão varia mais de uma ordem de grandeza



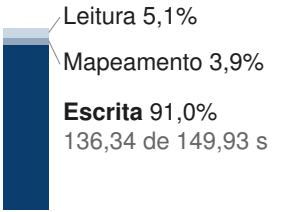
Vazão (*throughput*): linhas gravadas por segundo.

Cassandra: 1 GB de memória Java (*heap*) numa máquina virtual de cerca de 3,8 GB.

Fonte: Tabela 15, Figura 18 e Figura 20 do relatório; multifonte, em lote; fases e vazão com $N = 100.000$ e materialização; campanha sem instrumentação de memória.

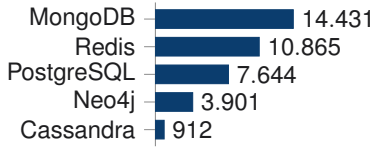
A escrita domina o tempo (QP1, QP3)

A escrita é 91,0% do tempo



Tempo por fase, em 100 mil linhas.

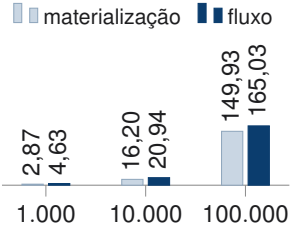
A vazão varia mais de uma ordem de grandeza



Vazão (*throughput*): linhas gravadas por segundo.

Cassandra: 1 GB de memória Java (*heap*) numa máquina virtual de cerca de 3,8 GB.

Tempo aproximadamente proporcional ao volume



Tempo total (s) por número de linhas.

Fonte: Tabela 15, Figura 18 e Figura 20 do relatório; multifonte, em lote; fases e vazão com N = 100.000 e materialização; campanha sem instrumentação de memória.

Multifonte × combinado: empate no tempo (QP2)

Multifonte *versus* combinado, 100 mil linhas, em lote

	Multifonte	Combinado
Tempo (s)		
campanha sem instrumentação de memória		
Total, materialização	149,93	161,94
Total, fluxo	165,03	181,42
Leitura, materialização	7,69	15,15
Pico de memória (MB)		
campanha com instrumentação de memória		
Materialização	950,7	1.009,5
Fluxo	75,9	79,8

Fonte: Tabela 17 do relatório; N = 100.000, em lote; tempos da campanha sem instrumentação de memória, picos da campanha com instrumentação.

Multifonte × combinado: empate no tempo (QP2)

Multifonte *versus* combinado, 100 mil linhas, em lote

	Multifonte	Combinado
Tempo (s)		
campanha sem instrumentação de memória		
Total, materialização	149,93	161,94
Total, fluxo	165,03	181,42
Leitura, materialização	7,69	15,15
Pico de memória (MB)		
campanha com instrumentação de memória		
Materialização	950,7	1.009,5
Fluxo	75,9	79,8

Tempo total: empate. O combinado é cerca de 8% (materialização) e 10% (fluxo) mais lento, abaixo da incerteza de cerca de 20%.

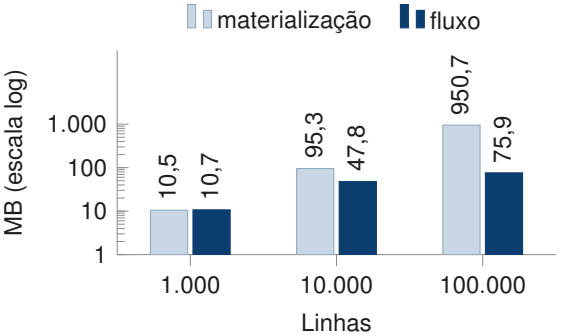
Leitura: o multifonte lê duas vezes mais rápido.

Memória: o multifonte ocupa de 5 a 6% menos.

O combinado tem 45% de células vazias e é separado por tipo de evento durante a leitura.

Fonte: Tabela 17 do relatório; $N = 100.000$, em lote; tempos da campanha sem instrumentação de memória, picos da campanha com instrumentação.

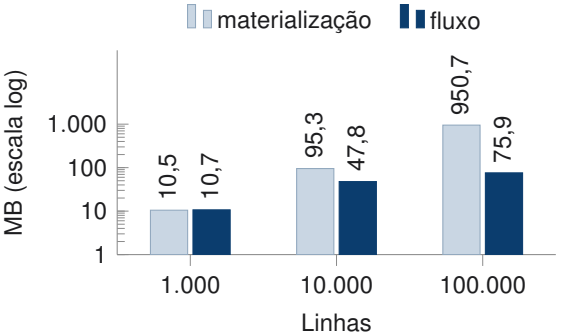
O fluxo achata o pico de memória (QP5, QP6)



Pico de memória por volume (escala log).

Fonte: Tabela 18 e Figura 19 do relatório; multifonte, em lote; campanha com instrumentação de memória.

O fluxo achata o pico de memória (QP5, QP6)



Pico de memória por volume (escala log).
Fonte: Tabela 18 e Figura 19 do relatório; multifonte, em lote; campanha com instrumentação de memória.

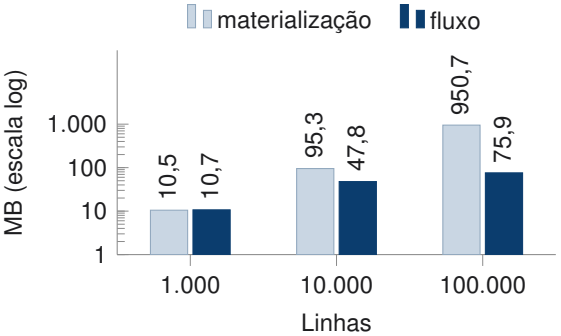
12,5×

menos memória em 100 mil linhas

Materialização: praticamente 90× mais memória para 100× mais dados.

Fluxo: o pico achata. Depende do tamanho do bloco, não do arquivo (QP6).

O fluxo achata o pico de memória (QP5, QP6)



Pico de memória por volume (escala log).
Fonte: Tabela 18 e Figura 19 do relatório; multifonte, em lote; campanha com instrumentação de memória.

12,5×
menos memória em 100 mil linhas

Materialização: praticamente 90× mais memória para 100× mais dados.

Fluxo: o pico achata. Depende do tamanho do bloco, não do arquivo (QP6).

Custo: +10% de tempo em 100 mil linhas (+61% em mil).
Fonte: Tabela 19 do relatório; campanha sem instrumentação de memória.

Roteiro

1 Introdução
2 Fundamentação e trabalhos relacionados
3 Proposta
4 Avaliação
5 Conclusão

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração impede o início da importação.	JSON Schema e consistência entre os arquivos antes de ler os CSVs; verificação dos SGBDs; validação cruzada completa no `--dry-run` e na GUI.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração impede o início da importação.	JSON Schema e consistência entre os arquivos antes de ler os CSVs; verificação dos SGBDs; validação cruzada completa no `--dry-run` e na GUI.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Seis questões de pesquisa (QP1–QP6); de 1.000 a 100.000 linhas; os cinco SGBDs; tempo e memória em campanhas separadas.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração impede o início da importação.	JSON Schema e consistência entre os arquivos antes de ler os CSVs; verificação dos SGBDs; validação cruzada completa no `--dry-run` e na GUI.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Seis questões de pesquisa (QP1–QP6); de 1.000 a 100.000 linhas; os cinco SGBDs; tempo e memória em campanhas separadas.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).	CLI com `--dry-run`, `--only` e `--execution`; GUI que monta e executa o comando, mantendo-o visível.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração impede o início da importação.	JSON Schema e consistência entre os arquivos antes de ler os CSVs; verificação dos SGBDs; validação cruzada completa no `--dry-run` e na GUI.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Seis questões de pesquisa (QP1–QP6); de 1.000 a 100.000 linhas; os cinco SGBDs; tempo e memória em campanhas separadas.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).	CLI com `--dry-run`, `--only` e `--execution`; GUI que monta e executa o comando, mantendo-o visível.

Uma ferramenta importa o mesmo CSV para cinco SGBDs de paradigmas diferentes.

Resolve × em aberto

O artigo trata de **evolução de esquemas**; este trabalho, de **importação**.
Três questões do artigo, levadas para a importação (ARAUJO; SILVA; MELLO, 2025):

Questão do artigo	Na importação, com a Polyglot Import CSV	Situação
Gerar instruções adequadas a cada SGBD	Um só arquivo de importação gera a escrita de cada SGBD: SQL, CQL (Cassandra), Cypher (Neo4j), comandos do Redis e documentos do MongoDB.	**resolve**
Estimar o efeito das modificações antes que sejam executadas	Validação prévia e `--dry-run`: contagens por SGBD e por entidade, sem gravar.	**resolve**

Resolve × em aberto

O artigo trata de **evolução de esquemas**; este trabalho, de **importação**.
Três questões do artigo, levadas para a importação (ARAUJO; SILVA; MELLO, 2025):

Questão do artigo	Na importação, com a Polyglot Import CSV	Situação
Gerar instruções adequadas a cada SGBD	Um só arquivo de importação gera a escrita de cada SGBD: SQL, CQL (Cassandra), Cypher (Neo4j), comandos do Redis e documentos do MongoDB.	**resolve**
Estimar o efeito das modificações antes que sejam executadas	Validação prévia e `--dry-run`: contagens por SGBD e por entidade, sem gravar.	**resolve**
Garantir atomicidade (tudo ou nada)	Grava um SGBD por vez, sem transação comum: uma falha no meio deixa uma carga parcial.	**em aberto**

Trabalhos futuros

Um sexto paradigma

Relacional	Chave-valor	Documento	Colunar	Grafo	Motor de busca
PostgreSQL	Redis	MongoDB	Cassandra	Neo4j	Elasticsearch

O Elasticsearch entraria como um novo destino no mesmo núcleo de importação.

Outras frentes

- **Edição da configuração na GUI**, com validação durante a digitação.
- **Milhões de linhas**, com SGBDs dedicados e ajustados.
- **Coerção configurável**: o usuário define como tratar valores malformados.
- **Outros formatos de entrada**: TSV e planilhas Excel.



Universidade Federal de Santa Catarina · Centro Tecnológico
Departamento de Informática e Estatística · Ciência da Computação

Obrigado! Perguntas?

Lucas Bueno Cesario

Falha no meio da importação

- **Antes de ler os CSVs**, verifica se todos os SGBDs de destino respondem. Um fora do ar interrompe tudo.
- **Um SGBD por vez**, em lotes: PostgreSQL, MongoDB, Cassandra, Redis, Neo4j.
- **Sem atomicidade entre SGBDs** (em aberto, slide 23): não há transação comum nem desfazer (*rollback*).
- **Falha no meio**: os SGBDs anteriores ficam gravados, o que falhou fica parcial, os seguintes ficam vazios.
- **Reexecução** com `--only`: com chave, PostgreSQL, Cassandra, Redis e Neo4j não duplicam. No MongoDB, sem `_id` mapeado, duplica documentos; com `_id`, para na chave repetida.

Coerção de tipos

O CSV é lido como **texto**. O tipo de cada coluna é **inferido** dos próprios valores, nesta ordem:

1. Booleano
todos os valores são `true` ou `false`

→

2. Número
todos são inteiros ou reais

→

3. Data e hora
mais de 85% são datas

→

4. Texto
nenhum dos casos anteriores

- Célula vazia vira nulo em número, booleano e data; em texto, fica vazia.
- Numa coluna de datas, valor inválido vira nulo, com aviso no log.
- Em fluxo, o tipo sai do primeiro bloco (8.192 linhas). Depois dele, número inválido fica texto, com aviso; booleano inválido vira falso, sem aviso.
- O tipo declarado (`db_type`) cria a coluna, mas não converte o valor. Uma coluna `BIGINT` do PostgreSQL pode recusar um texto e parar a importação.
- No PostgreSQL, linha com chave vazia é descartada, e chave repetida não duplica.

Política de coerção configurável: trabalho futuro (Cap. 6 do relatório).

Por que configurar à mão?

Mapeamento declarativo
o que a ferramenta faz hoje

- Quem projeta decide que dado vai para cada SGBD, com chaves, partições e arestas.
- O arquivo pode ser revisto, e o `--dry-run` o valida sem gravar.
- A mesma configuração reproduz a mesma carga.
- Parte já é automática: sem `columns`, entram todas as colunas, com tipos inferidos.

Mapeamento inferido
por um modelo de linguagem (LLM)

- Poupa a escrita inicial do arquivo.
- O CSV não diz como a aplicação consulta os dados. É isso que decide o SGBD de cada dado.
- A proposta ainda precisa de revisão.

Complementares: um LLM poderia propor o arquivo de importação, e o `--dry-run` o conferiria sem gravar. Caminho possível, fora do escopo deste trabalho.

Importadores nativos: o que cada um exige

SGBD	Importador	O que exige
PostgreSQL	`COPY`	colunas na ordem do CSV; arquivo na máquina do servidor; permissão de superusuário
Redis	`redis-cli --pipe`	arquivo prévio com os comandos (`SET chave valor`); resumo só com os totais de erros e respostas
MongoDB	`mongoimport`	banco, coleção e formato por parâmetros; uma coleção por comando
Cassandra	`cqlsh`, `sstableloader`	tipos e colunas iguais aos da tabela; o `sstableloader` exige SSTables (formato próprio)
Neo4j	`neo4j-admin import`	CSVs separados para nós e para relacionamentos; banco novo, vazio
Polyglot Import CSV		**o mesmo CSV nos cinco SGBDs: um arquivo de importação, validação prévia e `--dry-run`**

Comparação qualitativa: o Cap. 5 não mede esses importadores. Fonte: §3.1 do relatório.

Cassandra e o ambiente

Escrita por SGBD

SGBD	Linhas	Tempo (s)	Linhas/s
Cassandra	100.000	**109,61**	**912**
Neo4j	51.602	13,23	3.901
PostgreSQL	62.167	8,13	7.644
Redis	49.558	4,56	10.865
MongoDB	11.609	0,80	14.431

O Cassandra grava mais linhas e mais devagar: **109,6 s dos 136,3 s** da escrita, cerca de 80%.

Fonte: Tabela 16 do relatório; N = 100.000, multifonte, materialização, em lote; campanha sem instrumentação de memória.

Ambiente (§5.2)

Uma máquina, 8 GB de memória, os cinco SGBDs em contêineres Docker. Cassandra: *heap* de 1 GB numa máquina virtual Docker de cerca de 3,8 GB.

Leitura do relatório (§5.4.1)

As 912 linhas/s refletem o *heap* pequeno: restrição do ambiente, não do SGBD.

Escala: trabalho futuro

SGBDs dedicados e ajustados; escrita do Cassandra além da concorrência atual.

Ameaças à validade

De construção
mede o que se quer?

- Tempo com incerteza de cerca de 20%: diferenças menores não são interpretadas.
- Memória estável: variação abaixo de 0,1%.
- Tempo e memória vêm de campanhas separadas.

Interna
o efeito vem do fator?

- Gerador determinístico e SGBDs limpos antes de cada repetição.
- Volume moderado: até 100.000 linhas.
- Sem caches quentes nem execuções concorrentes.

Externa
vale fora deste ambiente?

- Uma máquina, Windows, cinco SGBDs em contêineres com 8 GB: não é produção.
- Cassandra limitado pelo *heap* de 1 GB.
- Dados sintéticos de um só domínio, o e-commerce (§5.1.1).

Fonte: §5.4.1 do relatório.

Referências

ARAUJO, L. N.; SILVA, H. A. B. d.; MELLO, R. d. S. Uma revisão sistemática simplificada e análise de estudos sobre evolução de esquemas em aplicações de persistência poliglota. In: *Anais da Escola Regional de Banco de Dados (ERBD)*. [S.l.]: Sociedade Brasileira de Computação, 2025. DOI: 10.5753/erbd.2025.7322.

PEREIRA, F. d. O.; GUERRA, E.; ROSA, R. R. A framework model for supporting transparent polyglot persistence with a unified API and extensible for different database types. In: *Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS)*. [S.l.]: SciTePress, 2025. p. 109–120. DOI: 10.5220/0013369300003929.

SACHDEVA, S.; VASAVA, J. Plugging and playing with variety of data using multi-model database and polyglot persistence. In: *2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*. [S.l.]: IEEE, 2024. p. 1–8. DOI: 10.1109/ICEEICT61591.2024.10718481.